专升本 **Java** 程序设计模拟卷-卷 **12**

fifine-专升本

2026 年 3 月 2 日

fifine专升本

1 一、单项选择题深度剖析

1. 在 **Java** 中，以下哪个选项可以正确编译？

A. `float f = 3.14;`

B. `byte b = 200;`

C. `char c = "a";`

D. `long l = 100L;`

答案与深度解析 答案：**D.** `long l = 100L;`

【核心认知框架】

- 题目本质：考查 Java 强类型系统中的字面量（Literal）默认类型机制与隐式类型转换（Narrowing/Widening Conversion）规则。
- 关键区分点：区分编译期常量折叠范围与运行时溢出，区分字面量后缀修饰符的硬性规范。
- 命题人意图：测试考生对 JLS 基本数据类型规范的掌握，淘汰只靠 IDE 提示写代码的" 语法盲"。

【选项原理深度拆解】

- **A.** `float f = 3.14;` —错误（向下转型缺失）
  - 字节码铁证：

```
// 尝试编译: float f = 3.14;
// javac 报错: incompatible types: possible lossy conversion from double to float
```

  JVM 保障机制：Java 中的浮点数字面量（如 3.14）默认类型是 `double`（64 位）。将其赋值给 `float`（32 位）会丢失精度，JLS §5.2 严禁未显式强转的向下转型（Narrowing Primitive Conversion）。
  - 修正规则：必须添加后缀 `f` 或 `F`，即 `float f = 3.14f;`。
- **B.** `byte b = 200;` —错误（常量折叠越界）
  - 底层机制解析：
    * JLS §5.2 规定：如果右侧是 `int` 类型的常量表达式，且值在左侧类型的范围内，允许隐式转换。
    * `byte` 的取值范围是 -128 到 127（8 位有符号整数）。200 超出了此范围。
    * 编译器在 AST（抽象语法树）阶段进行常量折叠时，直接阻断编译：`possible loss of precision`。
  - 易错点：如果写成 `byte b = (byte)200;`，编译通过，但运行时值会变成

fifine专升本

-56（二进制截断）。

- **C.** `char c = "a";` —错误（引用类型与基本类型错乱）
  - 内存模型解析：
    - "a" 是 String 对象（存放于堆内存的字符串常量池），类型是引用类型（Reference Type）。
    - char 是基本数据类型（Primitive Type），占用 16 位（2 字节），存储 Unicode 码位。
    - JLS §3.10.4 规定，字符字面量必须使用单引号 'a' 闭合。
- **D.** `long l = 100L;` —正确（规范的向上转型声明）
  - 字节码铁证：

```
long l = 100L;
// javap -c 输出:
// 0: ldc2_w #2 // 将long型常量100从常量池压入操作数栈
// 3: lstore_1 // 存入局部变量表索引1
```

  - 初始化规则：后缀 L 显式指示编译器将其作为 64 位整型处理（小写 l 极易与数字 1 混淆，阿里 Java 开发规范强制要求用大写 L）。

【应背诵知识点（命题人思维版）】

(a) 字面量默认类型：整数默认 int，浮点数默认 double。

(b) 隐式转换特权：byte/short/char 接受 int 常量赋值时，只要不越界，免强转。

(c) 高频陷阱清单：
  - 陷阱 1：`float f = 1.0;` → 漏写 f 必报错。
  - 陷阱 2：`long l = 2147483648;` → 报错！因为右侧默认是 int，超出了 int 的最大值。必须写成 2147483648L。

fffine专升本

2. 关于静态代码块，以下说法正确的是？

A. 静态代码块在每次创建对象时执行

B. 静态代码块在类加载时执行一次

C. 静态代码块中可以访问非静态成员

D. 静态代码块可以有多个，执行顺序不确定

答案与深度解析 答案：**B.** 静态代码块在类加载时执行一次

【核心认知框架】

- 题目本质：考查 JVM 类加载机制（Class Loading Subsystem）中的初始化阶

段（Initialization）行为。

- 关键区分点：区分类级别生命周期（<clinit>）与对象级别生命周期（<init>）。

【选项原理深度拆解】

- **A.** 在每次创建对象时执行—错误（混淆了构造块）
  - 机制解析：每次 new 对象时执行的是非静态的“构造代码块”和构造方法（JVM 中的 <init> 方法）。静态块属于类，不跟随实例触发。
- **B.** 在类加载时执行一次—正确（**<clinit>** 线程安全机制）
  - 字节码铁证：

```java
class Test {
    static int x;
    static { x = 10; }
}
// javap -c 输出发现，static块被编译器收集到了 <clinit> 方法中：
// static {};
// Code:
// 0: bipush 10
// 2: putstatic #2 // Field x:I
// 5: return
```

  - **JVM** 规范依据：JVMS §5.5 规定，静态块被编译为 <clinit>() 方法。JVM 在类的初始化阶段（触发条件如 new、访问静态变量、反射等）执行它。虚拟机会保证 <clinit>() 在多线程环境中被正确加锁、同步，确保全局只执行一次。
- **C.** 可以访问非静态成员—错误（内存语义冲突）
  - 内存影响：类加载时，对象可能根本还未创建，堆内存中不存在实例变量。静态块存储在方法区（元空间），它没有 this 指针，因此强行访问实例成员会直接触发编译期错误。
- **D.** 有多个时执行顺序不确定—错误（严格按文本顺序）
  - 初始化规则：JLS 明确规定，编译器收集静态变量赋值语句和静态块时，严格按照源文件中的文本出现顺序（**Top-to-Bottom**）合并到 <clinit>() 中。

【命题趋势与实战策略】

- 考场秒杀口诀：”静（态）只一次，顺（序）理成章；非静随对象，无 this 不见光”。

tfine专升本

2 二、判断题（底层真伪辩证）

1. 泛型类型参数在编译时被擦除，因此运行时无法判断 **List** 中存储的实际类型。

答案与深度解析 答案：✓ （正确）

【核心认知框架】

- 题目本质：考查 Java 泛型的核心设计哲学——类型擦除（Type Erasure），及与 C++ 模板机制的区别。
- 命题人意图：筛选出真正理解 JVM 向下兼容性历史包袱的高阶开发者。

【原理论证与字节码拆解】

- 正向论证：编译期擦除机制
  - 字节码铁证：

```java
List<String> list = new ArrayList<>();
list.add("Java");
String s = list.get(0);
// 反编译后的字节码（等同于泛型擦除后的真身）：
// List list = new ArrayList();
// list.add("Java");
// String s = (String) list.get(0); // 编译器偷偷塞入的 checkcast
    指令
```

  - **JVM** 规范依据：JLS §4.6，由于 Java 5 才引入泛型，为了保证旧版 JVM 能继续运行新版 Class 文件，所有泛型参数 `<T>` 在编译成字节码时都被替换为它的上界（默认是 `Object`）。
- 反向验证：运行时的无力感
  - 无法判断的铁证：在运行时执行 `if (list instanceof List<String>)` 会直接报编译错误：`Cannot perform instanceof check against parameterized type`。因为内存里根本没有 `List<String>` 这个类，只有 `List` 接口的实现。
- 高阶延伸（冲击满分）：
  - 例外情况：虽然运行时对象不知道自己的泛型，但类的元数据（签名）保留了泛型信息。可以通过反射（`ParameterizedType`）获取类的泛型父类/接口信息（这也是 Gson/Fastjson 等框架反序列化的底层依赖）。

fine专升本

3 三、程序运行结果题（JVM 执行栈追踪）

1. 写出下列程序的输出结果：

```java
public class Test1 {
    public static void main(String[] args) {
        int x = 10;
        int y = x++ + ++x + x--;
        System.out.println(x);
        System.out.println(y);
    }
}
```

答案与深度解析 答案：**11** **34**

【核心认知框架】

- 题目本质：考查 JVM 操作数栈（Operand Stack）与局部变量表（Local Variable Table）的交互，特别是 `iinc`（自增操作）与压栈指令执行的先后顺序。
- 易错点：将人的思维代入机器思维，混淆了”变量现在的值”与”参与计算的提取值”。

【JVM 操作数栈微观执行拆解】

- 字节码铁证分析（表达式：`y = x++ + ++x + x--`）：

  步骤 **1.** 初始化：`int x = 10;`
  - 局部变量表：`x=10`

  步骤 **2.** 执行第一部分：`x++`
  - 指令 `iload_1`：将局部变量表中的 10 压入操作数栈。← 栈内目前是 **10**
  - 指令 `iinc 1, 1`：局部变量表中的 x 直接加 1，变成 **11**。

  步骤 **3.** 执行第二部分：`++x`
  - 指令 `iinc 1, 1`：局部变量表中的 x 直接加 1，变成 **12**。
  - 指令 `iload_1`：将局部变量表中新的 12 压入操作数栈。← 栈内目前是 **10, 12**
  - 执行加法 `iadd`：弹出 10 和 12 相加，结果 **22** 压栈。

  步骤 **4.** 执行第三部分：`x--`
  - 指令 `iload_1`：将局部变量表中的 12 压入操作数栈。← 栈内目前是 **22, 12**
  - 指令 `iinc 1, -1`：局部变量表中的 x 直接减 1，变成 **11**。

  步骤 **5.** 最终加和赋给 **y**
  - 执行加法 `iadd`：弹出 22 和 12 相加，结果 **34**。

fifine专升本

  - 指令 `istore_2`：将 **34** 存入局部变量表 `y` 中。
- 最终状态结论：
  - 内存中 `x` 经历了：$10 \to 11 \to 12 \to 11$。所以 `x = 11`。
  - 操作数栈计算：$10 + 12 + 12 = 34$。所以 `y = 34`。

【命题陷阱破解】

- 高频错误计算法：$10 + 11 + 10 = 31$。（完全弄反了后置与前置的压栈时机）。
- 考场秒杀口诀：
  - `++` 在后（`x++`）：先提值，后自增（先记下当前值参与算术，转头变量立刻加 1）。
  - `++` 在前（`++x`）：先自增，后提值（变量先加 1，然后拿出新值参与算术）。

fifine专升本

4 四、编程题（企业级架构落地示范）

1. 设计一个”员工”类及其子类（**20** 分）

标准企业级答案与多态深度解析 【核心考点对齐】

- 考察 abstract 抽象类的顶层抽象能力。
- 考察 ArrayList 配合 for-each 进行动态绑定（Dynamic Binding）的多态实战。

【严谨高分源码（含规范级注释）】

```java
import java.util.ArrayList;

// 1. 顶层抽象类 (符合 OCP 开闭原则)
abstract class Employee {
    private String name;
    private String id; // 工号可能含字母，用String更严谨

    public Employee(String name, String id) {
        this.name = name;
        this.id = id;
    }

    // Getter/Setter 略(防篇幅过长，考试须写全)
    public String getName() { return name; }

    // 核心抽象方法，推迟到子类实现
    public abstract double calculateSalary();
}

// 2. 子类1: 固定薪资员工
class SalariedEmployee extends Employee {
    private double monthlySalary;

    public SalariedEmployee(String name, String id, double monthlySalary) {
        super(name, id); // 强制调用父类构造，初始化继承状态
        this.monthlySalary = monthlySalary;
    }

    @Override // 必须标注，利用编译器进行重写规则检查
    public double calculateSalary() {
        return monthlySalary;
```

fifine专升本

```java
    }
}

// 3. 子类2: 小时工
class HourlyEmployee extends Employee {
    private double hoursWorked;
    private double hourlyRate;

    public HourlyEmployee(String name, String id, double hoursWorked, double hourlyRate) {
        super(name, id);
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    @Override
    public double calculateSalary() {
        return hoursWorked * hourlyRate;
    }
}

// 4. 测试类: 多态威力展示
public class Main {
    public static void main(String[] args) {
        // 面向父类编程: 容器装的是父类型
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new SalariedEmployee("张三", "S001", 8000));
        list.add(new HourlyEmployee("李四", "H001", 120, 50));

        // JLS多态核心: 编译看左边(Employee), 运行看右边(具体子类)
        for (Employee emp : list) {
            // JVM底层通过 vtable (虚方法表) 执行动态分派 invokevirtual
            double salary = emp.calculateSalary();
            System.out.println(emp.getName() + " 的工资是: " + salary);
        }
    }
}
```

【判卷得分点（讲师批卷视角）】

ffine专升本

1. 封装性（**4** 分）：属性是否设为 `private`，并通过 public 的 getter/setter 访问。未封装直接扣 2 分。
2. 抽象修饰符（**4** 分）：类名和 `calculateSalary` 方法前是否有 `abstract` 关键字。
3. 构造链调用（**5** 分）：子类构造器第一行是否显式使用 `super(name, id);`。如果没有，由于父类没有无参构造，编译将直接崩溃。
4. 多态遍历（**7** 分）：`ArrayList<Employee>` 的泛型声明与循环中 `emp.calculateSalary()` 的动态绑定调用。此处最忌讳写成冗长的 `if (emp instanceof SalariedEmployee)`。

【**JVM** 虚拟机底层视角分析】当执行 `emp.calculateSalary()` 时，JVM 使用字节码指令 `invokevirtual`。

- 方法调用解析：JVM 去堆内存中找到 `emp` 实际指向的对象（例如 `HourlyEmployee` 的实例）。
- 虚方法表（**vtable**）寻址：在方法区（元空间）中查询该对象对应类的虚方法表，直接定位到子类重写后的方法入口指针执行。这实现了无条件的时间复杂度 **O(1)** 的多态分发，是 Java 面向对象性能优化的核心。

fifine专升本